

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Atividade 4:

Atividade 4 – Ingestão e ETL com linguagem de programação (Python + Spark + SQL)

- Utilizar linguagem de programação Python para ingestão e tratamento de dados. Para processo de transformação deve ser realizado via SQL com uma das seguintes ferramentas:
 - DBT (indicada)
 - SQL Mash – Alternativa
 - Coalesce – Alternativa
- Base final deve ser um banco de dados relacional já utilizado nos exercícios anteriores.
- O processamento SQL deve ser utilizado via ferramenta mais uma engine de SQL que pode ser o próprio Banco de Dados bem como um banco de dados em memória como Duckdb ou SQLite, ou qualquer outra alternativa.
- Gerar uma tabela final com os dados tratados e unidos em banco de dados:
 - O tratamento de dados deve ser realizado através da ferramenta escolhida para SQL.
- Adicionar as seguintes camadas de processamento, dentro do próprio banco de dados ou em disco local. A Camada Delivery deve obrigatoriamente ter estar também no formato de uma tabela final dentro do banco de dados relacional:
 - RAW – formato dos dados livre
 - Trusted – formato de dados em Parquet ou ORC or AVRO (indicado Parquet)
 - Delivery – formato de dados em Parquet ou ORC or AVRO (indicado Parquet)

Decisões: PostgreSQL, DBT e Spark local

Link do GitHub:

Docker rodando:

```
(base) camil@MacBook-Air eEDB-022 % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS         NAMES
e85ab10dcdd0   postgres:16   "docker-entrypoint.s..." 9 seconds ago  Up 9 seconds
0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp  pg-eedb022-a4
(base) camil@MacBook-Air eEDB-022 %
```

Estrutura de pastas criadas dentro do repositório:

```
atividade_04
├── data
│   ├── delivery
│   └── trusted
├── jars
│   └── postgresql-42.7.4.jar
├── scripts
├── Dados
│   ├── Bancos
│   │   └── EnquadramentoInicia_v2.tsv
│   ├── Empregados
│   │   ├── glassdoor_consolidado_join_match_less_v2.csv
│   │   └── glassdoor_consolidado_join_match_v2.csv
│   └── Reclamacoes
│       ├── 2021_tri_01.csv
│       ├── 2021_tri_02.csv
│       ├── 2021_tri_03.csv
│       ├── 2021_tri_04.csv
│       ├── 2022_tri_01.csv
│       ├── 2022_tri_02_nao_ha_dados.csv
│       ├── 2022_tri_03.csv
│       └── 2022_tri_04.csv
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Criando schemas no banco:

```
(base) camil@MacBook-Air atividade_04 % docker exec -it pg-eedb022-a4 psql -U postgres -d eedb022_a4 -i
c "CREATE SCHEMA raw; CREATE SCHEMA trusted; CREATE SCHEMA delivery;"
CREATE SCHEMA
CREATE SCHEMA
CREATE SCHEMA
(base) camil@MacBook-Air atividade_04 %
```

Script para leitura e ingestão da camada Raw:

```
atividade_04 > scripts > ingest_raw.py > ...
1  from pyspark.sql import SparkSession
2
3  JDBC_JAR = "jars/postgresql-42.7.4.jar"
4
5  PG_URL = "jdbc:postgresql://localhost:5433/eedb022_a4"
6  PG_PROPS = {
7      "user": "postgres",
8      "password": "postgres",
9      "driver": "org.postgresql.Driver"
10 }
11
12 DADOS_DIR = "../Dados"
13
14 spark = SparkSession.builder \
15     .appName("ingestao_raw_atividade4") \
16     .config("spark.jars", JDBC_JAR) \
17     .getOrCreate()
18
19 print("Lendo reclamações...")
20 trimestres = [
21     "2021_tri_01.csv", "2021_tri_02.csv", "2021_tri_03.csv", "2021_tri_04.csv",
22     "2022_tri_01.csv", "2022_tri_03.csv", "2022_tri_04.csv"
23 ]
24 caminhos = [f"{DADOS_DIR}/Reclamacoes/{arquivo}" for arquivo in trimestres]
25
26 df_reclamacoes = spark.read.csv(
27     caminhos,
28     sep=";",
29     encoding="ISO-8859-1",
30     header=True,
31     inferSchema=True
32 )
33 print(f"Reclamações: {df_reclamacoes.count()} linhas")
34
35 print("Lendo bancos (Enquadramento)...")
36 df_bancos = spark.read.csv(
37     f"{DADOS_DIR}/Bancos/EnquadramentoInicia_v2.tsv",
38     sep="\t",
39     header=True,
40     inferSchema=True
41 )
42 print(f"Bancos: {df_bancos.count()} linhas")
43
44 print("Lendo Glassdoor...")
45 df_glassdoor = spark.read.csv(
46     f"{DADOS_DIR}/Empregados/glassdoor_consolidado_join_match_v2.csv",
47     sep="|",
48     header=True,
49     inferSchema=True
50 )
51 print(f"Glassdoor: {df_glassdoor.count()} linhas")
52
53 print("Escrevendo no Postgres (schema raw)...")
54 df_reclamacoes.write.jdbc(PG_URL, "raw.reclamacoes", mode="overwrite", properties=PG_PROPS)
55 df_bancos.write.jdbc(PG_URL, "raw.bancos", mode="overwrite", properties=PG_PROPS)
56 df_glassdoor.write.jdbc(PG_URL, "raw.empregados", mode="overwrite", properties=PG_PROPS)
57
58 print("Ingestão RAW concluída!")
59 spark.stop()
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Camada Raw ingerida:

```
Ingestão RAW concluída!
(base) camil@MacBook-Air atividade_04 % docker exec -it pg-eeedb022-a4 psql -U postgres -d eedb022_a4 -c "SELECT COUNT(*) FROM raw.reclamacoes; SELECT COUNT(*) FROM raw.bancos; SELECT COUNT(*) FROM raw.empregados;"
count
-----
    918
(1 row)

count
-----
   1474
(1 row)

count
-----
     34
(1 row)
```

Inicializando o DBT para o tratamento dos dados:

```
(dbt_env) camil@MacBook-Air eedb022_dbt % dbt debug
16:19:35 Running with dbt=1.12.3
16:19:35 dbt version: 1.12.3
16:19:35 python version: 3.11.16
16:19:35 python path: /opt/anaconda3/envs/dbt_env/bin/python3.11
16:19:35 os info: macOS-15.7.4-arm64-arm-64bit
16:19:35 Using profiles dir at /Users/camil/.dbt
16:19:35 Using profiles.yml file at /Users/camil/.dbt/profiles.yml
16:19:35 Using dbt_project.yml file at /Users/camil/Documents/pós graduação/ingestão de dados/eEDB-022/atividade_04/eeedb022_dbt/dbt_project.yml
16:19:35 adapter type: postgres
16:19:35 adapter version: 1.11.0
16:19:35 Configuration:
16:19:35   profiles.yml file [OK found and valid]
16:19:35   dbt_project.yml file [OK found and valid]
16:19:35 Required dependencies:
16:19:35   - git [OK found]

16:19:35 Connection:
16:19:35   host: localhost
16:19:35   port: 5433
16:19:35   user: postgres
16:19:35   database: eedb022_a4
16:19:35   schema: trusted
16:19:35   connect_timeout: 10
16:19:35   role: None
16:19:35   search_path: None
16:19:35   keepalives_idle: 0
16:19:35   sslmode: None
16:19:35   sslcert: None
16:19:35   sslkey: None
16:19:35   sslrootcert: None
16:19:35   application_name: dbt
16:19:35   autocommit: False
16:19:35   retries: 1
16:19:35 Registered adapter: postgres=1.11.0
16:19:35 Connection test: [OK connection ok]

16:19:35 All checks passed!
(dbt_env) camil@MacBook-Air eedb022_dbt %
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Criando as models:

```
atividade_04 > eedb022_dbt > models >  trusted_bancos.sql  
1  SELECT DISTINCT  
2      LPAD(CAST("CNPJ" AS TEXT), 8, '0') AS cnpj,  
3      UPPER(TRIM("Nome")) AS nome,  
4      "Segmento" AS segmento  
5  FROM raw.bancos  
6  WHERE "CNPJ" IS NOT NULL AND "CNPJ" != 0  
7
```

Padronizei os CNPJs para 8 dígitos completos, recuperando os zeros à esquerda que se perderam por estarem armazenados como número. Removi registros com CNPJ inválido (placeholder zero), normalizei os nomes dos bancos (removendo espaços e padronizando em maiúsculo) e eliminei linhas duplicadas.

```
atividade_04 > eedb022_dbt > models >  trusted_reclamacoes.sql  
1  SELECT DISTINCT  
2      "Ano" AS ano,  
3      "Trimestre" AS trimestre,  
4      LPAD(TRIM("CNPJ IF"), 8, '0') AS cnpj,  
5      "Instituição financeira" AS instituicao,  
6      "Quantidade total de reclamações" AS qtd_reclamacoes  
7  FROM raw.reclamacoes  
8  WHERE "CNPJ IF" IS NOT NULL AND TRIM("CNPJ IF") != ''  
9
```

Removi os registros sem CNPJ próprio (linhas de conglomerados que não têm identificação individual), padronizei os CNPJs para 8 dígitos completos, excluí colunas irrelevantes ou corrompidas geradas na ingestão, e eliminei linhas duplicadas.

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

```
atividade_04 > eedb022_dbt > models > trusted_empregados.sql
1  SELECT DISTINCT
2      UPPER(TRIM("Nome")) AS nome,
3      "employer_name" AS employer_name,
4      "Geral" AS nota_geral,
5      "Remuneração e benefícios" AS nota_remuneracao,
6      "Recomendam para outras pessoas(%)" AS pct_recomendam
7  FROM raw.empregados
8  WHERE "Nome" IS NOT NULL AND TRIM("Nome") != ''
9
```

Normalizei os nomes dos bancos (removendo espaços e padronizando em maiúsculo) para garantir compatibilidade no cruzamento com as demais fontes, removi registros sem nome de banco preenchido, e eliminei linhas duplicadas.

```
atividade_04 > eedb022_dbt > models > delivery_final.sql
1  WITH reclamacoes_agg AS (
2      SELECT
3          cnpj,
4          SUM(qtd_reclamacoes) AS total_reclamacoes
5      FROM {{ ref('trusted_reclamacoes') }}
6      GROUP BY cnpj
7  )
8
9  SELECT
10     b.cnpj,
11     b.nome,
12     b.segmento,
13     COALESCE(r.total_reclamacoes, 0) AS total_reclamacoes,
14     e.nota_geral,
15     e.nota_remuneracao,
16     e.pct_recomendam
17  FROM {{ ref('trusted_bancos') }} b
18  LEFT JOIN reclamacoes_agg r ON b.cnpj = r.cnpj
19  LEFT JOIN {{ ref('trusted_empregados') }} e ON b.nome = e.nome
20
```

Uni as três fontes tratadas (bancos, reclamações e avaliações Glassdoor) por CNPJ e nome padronizados, agreguei o total de reclamações por instituição, e tratei os bancos sem reclamações registradas para exibir valor zero em vez de vazio.

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

```
(dbt_env) camil@MacBook-Air eedb022_dbt %  
  
(dbt_env) camil@MacBook-Air eedb022_dbt % dbt run  
16:38:03 Running with dbt=1.12.3  
16:38:03 Registered adapter: postgres=1.11.0  
16:38:03 Found 4 models, 477 macros  
16:38:03  
16:38:03 Concurrency: 4 threads (target='dev')  
16:38:03  
16:38:03 1 of 4 START sql table model trusted.trusted_bancos ..... [RUN]  
16:38:03 2 of 4 START sql table model trusted.trusted_empregados ..... [RUN]  
16:38:03 3 of 4 START sql table model trusted.trusted_reclamacoes ..... [RUN]  
16:38:03 2 of 4 OK created sql table model trusted.trusted_empregados ..... [SELECT 34 in 0.09s]  
16:38:03 3 of 4 OK created sql table model trusted.trusted_reclamacoes ..... [SELECT 437 in 0.09s]  
16:38:03 1 of 4 OK created sql table model trusted.trusted_bancos ..... [SELECT 1472 in 0.09s]  
16:38:03 4 of 4 START sql table model trusted.delivery_final ..... [RUN]  
16:38:03 4 of 4 OK created sql table model trusted.delivery_final ..... [SELECT 1472 in 0.03s]  
16:38:03  
16:38:03 Finished running 4 table models in 0 hours 0 minutes and 0.27 seconds (0.27s).  
16:38:03  
16:38:03 Completed successfully  
16:38:03  
16:38:03 Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=4  
(dbt_env) camil@MacBook-Air eedb022_dbt %
```

Fui validar no banco e vi que os acentos vieram quebrados, então usei a mesma forma de correção que usamos na atividade 2: batemos os nomes com a API do Banco do Brasil e colocamos esse novo resultado na tabela raw.depara_bcb:

```
atividade_04 > scripts > depara_bcb.py > ...  
1 import requests  
2 import psycopg2  
3  
4 url = "https://olinda.bcb.gov.br/olinda/servico/BcBase/versao/v2/odata/EntidadesSupervisionadas(dataBase=@dataBase)?@dataBase='12-31-2024'&$top=1000&$format=json"  
5  
6 response = requests.get(url)  
7 dados = response.json()["value"]  
8  
9 depara = {}  
10 for registro in dados:  
11     cnpj = registro.get("codigoCNPJ8")  
12     nome = registro.get("nomeEntidadeInteresse")  
13     if cnpj and nome:  
14         depara[cnpj] = nome  
15  
16 print(f"Total de CNPJs no de-para: {len(depara)}")  
17  
18 conn = psycopg2.connect(  
19     host="localhost",  
20     port=5433,  
21     user="postgres",  
22     password="postgres",  
23     dbname="eedb022_a4"  
24 )  
25 cur = conn.cursor()  
26  
27 cur.execute("DROP TABLE IF EXISTS raw.depara_bcb")  
28 cur.execute("CREATE TABLE raw.depara_bcb (cnpj TEXT, nome TEXT)")  
29  
30 for cnpj, nome in depara.items():  
31     cur.execute("INSERT INTO raw.depara_bcb (cnpj, nome) VALUES (%s, %s)", (cnpj, nome))  
32  
33 conn.commit()  
34 cur.close()  
35 conn.close()  
36  
37 print("De-para carregado em raw.depara_bcb")
```

Agora sim:

```
----- nome_completo -----  
COOPERATIVA DE ECONOMIA E CRÉDITO MÚTUO DOS EMPREGADOS DO GRUPO TELEFÔNICA  
COOPERATIVA DE CRÉDITO SICOOB ITAPAGIPE LTDA – SICOOB ITAPAGIPE  
COOPERATIVA DE CRÉDITO SICREDI PERNAMBUCRED  
COOPERATIVA DE CRÉDITO SICOOB MÉDIO OESTE  
COOPERATIVA DE CRÉDITO MÚTUO DOS SERVIDORES PÚBLICOS DA ESTÂNCIA TURÍSTICA DE RIBEIRÃO PIRES – CREDIRP
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

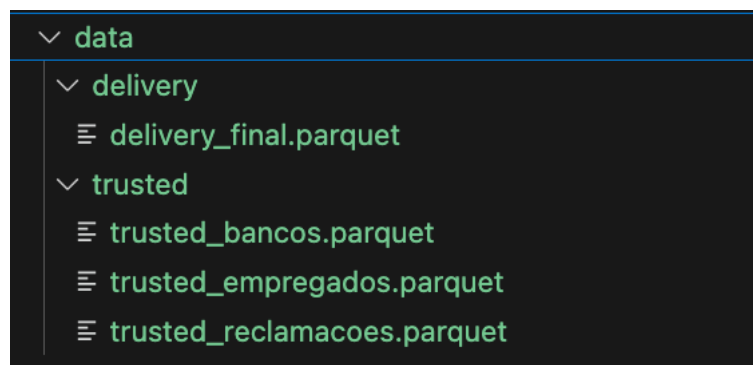
Agora vamos gerar o parquet das nossas camadas trusted e delivery:

```
atividade_04 > scripts > export_parquet.py > ...
1  import pandas as pd
2  from sqlalchemy import create_engine
3
4  engine = create_engine("postgresql://postgres:postgres@localhost:5433/eedb022_a4")
5
6  tabelas_trusted = ["trusted_bancos", "trusted_reclamacoes", "trusted_empregados"]
7
8  for tabela in tabelas_trusted:
9      df = pd.read_sql(f"SELECT * FROM trusted.{tabela}", engine)
10     df.to_parquet(f"data/trusted/{tabela}.parquet", index=False)
11     print(f"{tabela}: {len(df)} linhas exportadas")
12
13 df_delivery = pd.read_sql("SELECT * FROM trusted.delivery_final", engine)
14 df_delivery.to_parquet("data/delivery/delivery_final.parquet", index=False)
15 print(f"delivery_final: {len(df_delivery)} linhas exportadas")
```

Executado com sucesso:

```
/opt/anaconda3/bin/python3 scripts/export_parquet.py
trusted_bancos: 1472 linhas exportadas
trusted_reclamacoes: 437 linhas exportadas
trusted_empregados: 32 linhas exportadas
delivery_final: 1472 linhas exportadas
(dbt_env) camil@MacBook-Air atividade_04 %
```

Arquivos entraram na pasta:



Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Validando tabelas no banco:

